Cryptography Theory & Practice
CIS 6371

Instructor: **Dr. Attila A. Yavuz**
TA: **Sean Hernandez**

October 2026

Mini-Project: Signcryption Implementations [100 Points]

(Upload your solutions to the CANVAS)
Requirements: Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: Extra Credit should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.

Introduction

Signcryption is a cryptographic technique that integrates the functionalities of digital signature and public-key encryption into a single logical step. Compared to the traditional "signature-then-encryption" approach, signcryption achieves both confidentiality and authenticity with significantly reduced computational cost and communication overhead. This makes it particularly relevant for resource-constrained environments such as embedded devices, IoT communications, and mobile applications where efficiency is crucial.

In this mini-project, you will implement and analyze two signcryption schemes that follow the same overall structure but differ in their mathematical formulations for computing the signature component s and reconstructing the verification value w. Through these implementations, you will gain practical experience with core cryptographic concepts, modular arithmetic, secure key generation, and message integrity verification. You will also practice using cryptographic libraries (e.g., OpenSSL) to handle large integer operations and hash functions, ensuring that your implementations are consistent with the specified algorithms.

The project is divided into three main parts:

1. Implementation of the *Certificate Authority* for system parameter generation and key pair creation for Alice and Bob.
2. Implementation of the *Signcryption* and *Unsigncryption* procedures for each of the two scheme variants, ensuring correct confidentiality and authenticity according to the provided specifications.
3. Analytical questions on the Boyko–Preneel–Venkatesan (BPV) algorithm, a precomputation technique for accelerating exponentiation in modular groups.

By completing this assignment, you will reinforce your understanding of how cryptographic primitives are engineered, how small mathematical differences can impact performance and security, and how to translate theoretical designs into functional, verifiable code.

Comparison of Signcryption Schemes

The main differences between Signcryption 1st Example and Signcryption 2nd Example are in the mathematical formulations:

Operation	**Signcryption 1st Example**	**Signcryption 2nd Example**
Signature computation	$s = \frac{x}{r+x_a} \bmod q$	$s = \frac{x}{1+x_a \cdot r} \bmod q$
Verification computation	$w = (y_a \cdot g^r)^{s \cdot x_b} \bmod p$	$w = (g \cdot y_a^r)^{s \cdot x_b} \bmod p$

Table 1: Mathematical differences between the two signcryption schemes

These differences affect the security properties and efficiency of the schemes. Both schemes provide confidentiality, authenticity, and non-repudiation, but with different mathematical foundations.

1 Signcryption 1st Example [50 pt] (Due: *October 29th, 2026 – 11:59 pm*)

1.1 Signcryption Key Generation: Certificate Authority [15 pt]

Task Description

Implement the key generation algorithm in the file `KeyGen.c`. You **ONLY** need to implement the `Generate_Key_Pair()` function. This function is a critical component of the signcryption system that generates public and private key pairs for both Alice and Bob.

Function Signature

```
void Generate_Key_Pair(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM **private_key,
                       BIGNUM **public_key, unsigned char *seed, int seed_offset)
```

Parameters

- `p`: A large prime number (system parameter)
- `q`: A prime divisor of $p - 1$ (system parameter)
- `g`: A generator of order q in the multiplicative group of integers modulo p (system parameter)
- `private_key`: Output parameter for the generated private key
- `public_key`: Output parameter for the generated public key
- `seed`: Seed value for random number generation
- `seed_offset`: Offset value to create different keys for Alice and Bob

Example Implementation Structure

```
void Generate_Key_Pair(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM **private_key,
                       BIGNUM **public_key, unsigned char *seed, int seed_offset)
{
    // 1. Create a modified seed with the offset

    // 2. Use PRNG to generate random bytes

    // 3. Convert random bytes to BIGNUM for private key

    // 4. Ensure private key is in range [1, q-1]

    // 5. Calculate public key as g^x mod p
}
```

1.1 Key Generation – Instructions

Input Arguments

- 32 Bytes of Seed in a file named **"Seed.txt"**.
- System parameters in **"Parameters.txt"**

Certificate Authority (CA) Performs the following steps to generate system parameters and key pairs:

1. CA reads the seed from a file named **"Seed.txt"** containing 32 characters (32 Bytes).
2. CA reads the system parameters from **"Parameters.txt"**, (p, q, g) where:
   - p is a large prime (2048 bits)
   - q is a prime factor of $p - 1$
   - g is a generator with order q mod p
3. CA generates key pairs for Alice and Bob:
   - Private keys x_a and x_b are randomly chosen from $\{1, 2, \ldots, q - 1\}$
   - Public keys $y_a = g^{x_a} \bmod p$ and $y_b = g^{x_b} \bmod p$
4. CA writes the system parameters and key pairs to files:
   - Alice's key pair in **"AliceKeys.txt"**
   - Bob's key pair in **"BobKeys.txt"**

Terminal:

```
Compile the key generation part with the following command:
gcc KeyGen.c -lcrypto -o CertificateAuthority
Run your code with the following commands:
./CertificateAuthority Seed#.txt Parameters#.txt
```

1.2 Signcryption: Alice [20 pt]

Task Description

Implement the signcryption algorithm in the file `Sign.c`. You **ONLY** need to implement the `Signcrypt()` function. This function is responsible for signcrypting a message from Alice to Bob, combining both digital signature and encryption functionalities in a single operation.

Function Signature

```
void Signcrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, const unsigned char *message,
               size_t message_len, BIGNUM *alice_private_key, BIGNUM *bob_public_key,
               BIGNUM **r, BIGNUM **s, unsigned char **ciphertext, size_t *ciphertext_len,
               unsigned char *seed, size_t seed_len, int seed_offset)
```

Parameters

- `p, q, g`: System parameters
- `message`: The plaintext message to be signcrypted
- `message_len`: Length of the message
- `alice_private_key`: Alice's private key (x_a)
- `bob_public_key`: Bob's public key (y_b)
- `r, s`: Output parameters for the signature components
- `ciphertext`: Output parameter for the encrypted message
- `ciphertext_len`: Output parameter for the length of the ciphertext
- `seed`: Seed value for random number generation
- `seed_len`: Length of the seed
- `seed_offset`: Offset value for randomness generation

Example Implementation Structure

```
void Signcrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, const unsigned char *message,
               size_t message_len, BIGNUM *alice_private_key, BIGNUM *bob_public_key,
               BIGNUM **r, BIGNUM **s, unsigned char **ciphertext, size_t *ciphertext_len,
               unsigned char *seed, size_t seed_len, int seed_offset)
{
    // 1. Generate random k using PRNG with modified seed

    // 2. Compute w = y_b^k mod p (shared secret)

    // 3. Derive encryption key K_e = Hash_G(w)

    // 4. Encrypt the message using K_e

    // 5. Compute r = Hash_H(m, w)

    // 6. Compute s = x/(r + x_a) mod q

    // 7. Compute c = E_k(m) (encrypt the message)
}
```

1.2 Signcryption: Alice – Instructions

Input Arguments

- Message file **"Message.txt"**.
- 32 Bytes of Seed in a file named **"Seed.txt"**.
- System parameters file **"Params.txt"**.
- Alice's key pair file **"AliceKeys.txt"**.
- Bob's public key file **"BobPublicKey.txt"**.

Alice performs the signcryption as follows:

1. Alice reads the message from **"Message.txt"**.
2. Alice reads the system parameters and key information.
3. Alice performs the signcryption algorithm:
   (a) Pick random $x \in \{1, 2, \ldots, q-1\}$
   (b) Compute $w = y_b^x \bmod p$
   (c) Derive key $k = G(w)$ using hash function G
   (d) Compute $r = H(m, w)$ using hash function H
   (e) Compute $s = \frac{x}{r+x_a} \bmod q$ /* Specific to 1st Example */
   (f) Encrypt the message $c = E_k(m)$ using symmetric encryption
4. Alice writes the signcrypted message (c, r, s) to **"Signcryption.txt"**.
5. Alice writes the signcrypted message to **"SigncryptedMessage.txt"** for Bob to read.

Terminal:

```
Compile the signers part with the following command:
gcc Sign.c -lcrypto -o Alice
Run your code with the following commands:
./Alice Message#.txt Seed#.txt
```

1.3 Unsigncryption: Bob [15 pt]

Task Description

Implement the unsigncryption algorithm in the file `Verify.c`. You **ONLY** need to implement the `Unsigncrypt()` function. This function is responsible for verifying and decrypting a signcrypted message sent from Alice to Bob.

Function Signature

```
int Unsigncrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM *r, BIGNUM *s,
                const unsigned char *ciphertext, size_t ciphertext_len,
                BIGNUM *bob_private_key, BIGNUM *alice_public_key,
                unsigned char **message, size_t *message_len)
```

Parameters

- `p, q, g`: System parameters
- `r, s`: Signature components from the signcryption
- `ciphertext`: The encrypted message
- `bob_private_key`: Bob's private key (x_b)
- `alice_public_key`: Alice's public key (y_a)
- `message`: Output parameter for the decrypted message
- `message_len`: Output parameter for the length of the decrypted message
- **Return value:** 1 if verification succeeds, 0 if it fails

Example Implementation Structure

```
int Unsigncrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM *r, BIGNUM *s,
                const unsigned char *ciphertext, size_t ciphertext_len,
                BIGNUM *bob_private_key, BIGNUM *alice_public_key,
                unsigned char **message, size_t *message_len)
{
    // 1. Compute w = (y_a * g^r)^(s*x_b) mod p

    // 2. Derive encryption key k = Hash_G(w)

    // 3. Decrypt the ciphertext using D_k(c)

    // 4. Compute r' = Hash_H(m, w)

    // 5. Verify that r = r' and return result
}
```

1.3 Unsigncryption: Bob – Instructions

Input Arguments

- System parameters file **"Params.txt"**.
- Bob's key pair file **"BobKeys.txt"**.
- Alice's public key file **"AlicePublicKey.txt"**.

Bob performs the unsigncryption as follows:

1. Bob reads the signcrypted message (c, r, s) from **"SigncryptedMessage.txt"**.
2. Bob reads the system parameters and key information.
3. Bob performs the unsigncryption algorithm:
   (a) Compute $w = (y_a \cdot g^r)^{s \cdot x_b} \bmod p$ /* Specific to 1st Example */
   (b) Derive key $k = G(w)$ using hash function G
   (c) Decrypt the message $m = D_k(c)$ using symmetric decryption
   (d) Verify that $r = H(m, w)$ using hash function H
4. If verification succeeds, Bob writes "Signature is Valid" to **"Verification.txt"** and the decrypted message to **"DecryptedMessage.txt"**.
5. If verification fails, Bob writes "Verification Failed" to **"Verification.txt"**.

Terminal:

```
Compile the verifier part with the following command:
gcc Verify.c -lcrypto -o Bob
Run your code with the following commands:
./Bob
```

---

2 Signcryption 2nd Example [30 pt] (Due: *November 12th, 2026 – 11:59 pm*)

2.1 Signcryption Key Generation: Certificate Authority [Same as 1st Example]

Task Description

Implement the key generation algorithm in the file `KeyGen.c`. You **ONLY** need to implement the `Generate_Key_Pair()` function. This function is a critical component of the signcryption system that generates public and private key pairs for both Alice and Bob.

The key-generation function is the same as in Signcryption 1st Example.

Function Signature

```
void Generate_Key_Pair(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM **private_key,
                       BIGNUM **public_key, unsigned char *seed, int seed_offset)
```

Parameters

- `p`: A large prime number (system parameter)
- `q`: A prime divisor of $p - 1$ (system parameter)
- `g`: A generator of order q in the multiplicative group of integers modulo p (system parameter)
- `private_key`: Output parameter for the generated private key
- `public_key`: Output parameter for the generated public key
- `seed`: Seed value for random number generation
- `seed_offset`: Offset value to create different keys for Alice and Bob

Example Implementation Structure

```
void Generate_Key_Pair(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM **private_key,
                       BIGNUM **public_key, unsigned char *seed, int seed_offset)
{
    // 1. Create a modified seed with the offset

    // 2. Use PRNG to generate random bytes

    // 3. Convert random bytes to BIGNUM for private key

    // 4. Ensure private key is in range [1, q-1]

    // 5. Calculate public key as g^x mod p
}
```

2.1 Key Generation – Instructions

Input Arguments

- 32 Bytes of Seed in a file named **"Seed.txt"**.
- System parameters in **"Parameters.txt"**.

Certificate Authority (CA) Performs the following steps to generate system parameters and key pairs:

1. CA reads the seed from a file named **"Seed.txt"** containing 32 characters (32 Bytes).
2. CA uses the seed to generate system parameters (p, q, g) where:
   - p is a large prime (2048 bits)
   - q is a prime factor of $p - 1$
   - g is a generator with order q mod p
3. CA generates key pairs for Alice and Bob:
   - Private keys x_a and x_b are randomly chosen from $\{1, 2, \ldots, q - 1\}$
   - Public keys $y_a = g^{x_a} \bmod p$ and $y_b = g^{x_b} \bmod p$
4. CA writes the system parameters and key pairs to files:
   - System parameters in **"Params.txt"**
   - Alice's key pair in **"AliceKeys.txt"**
   - Bob's key pair in **"BobKeys.txt"**

Terminal:

```
Compile the key generation part with the following command:
gcc KeyGen.c -lcrypto -o CertificateAuthority
Run your code with the following commands:
./CertificateAuthority Seed#.txt Parameters#.txt
```

2.2 Signcryption: Alice [15 pt]

Task Description

Implement the signcryption algorithm in the file `Sign.c`. You **ONLY** need to implement the `Signcrypt()` function. This function is responsible for signcrypting a message from Alice to Bob, combining both digital signature and encryption functionalities in a single operation.

Function Signature

```
void Signcrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, const unsigned char *message,
               size_t message_len, BIGNUM *alice_private_key, BIGNUM *bob_public_key,
               BIGNUM **r, BIGNUM **s, unsigned char **ciphertext, size_t *ciphertext_len,
               unsigned char *seed, size_t seed_len, int seed_offset)
```

Parameters

- `p, q, g`: System parameters
- `message`: The plaintext message to be signcrypted
- `message_len`: Length of the message
- `alice_private_key`: Alice's private key (x_a)
- `bob_public_key`: Bob's public key (y_b)
- `r, s`: Output parameters for the signature components
- `ciphertext`: Output parameter for the encrypted message
- `ciphertext_len`: Output parameter for the length of the ciphertext
- `seed`: Seed value for random number generation
- `seed_len`: Length of the seed
- `seed_offset`: Offset value for randomness generation

Example Implementation Structure

```
void Signcrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, const unsigned char *message,
               size_t message_len, BIGNUM *alice_private_key, BIGNUM *bob_public_key,
               BIGNUM **r, BIGNUM **s, unsigned char **ciphertext, size_t *ciphertext_len,
               unsigned char *seed, size_t seed_len, int seed_offset)
{
    // 1. Generate random k using PRNG with modified seed

    // 2. Compute w = y_b^k mod p (shared secret)

    // 3. Derive encryption key K_e = Hash_G(w)

    // 4. Encrypt the message using K_e

    // 5. Compute r = Hash_H(m, w)

    // 6. Compute s = x/(1 + x_a * r) mod q

    // 7. Compute c = E_k(m) (encrypt the message)
}
```

2.2 Signcryption: Alice – Instructions

Input Arguments

- Message file **"Message.txt"**.
- 32 Bytes of Seed in a file named **"Seed.txt"**.
- System parameters file **"Params.txt"**.
- Alice's key pair file **"AliceKeys.txt"**.
- Bob's public key file **"BobPublicKey.txt"**.

Alice performs the signcryption as follows:

1. Alice reads the message from **"Message.txt"**.
2. Alice reads the system parameters and key information.
3. Alice performs the signcryption algorithm:
   (a) Pick random $x \in \{1, 2, \ldots, q-1\}$
   (b) Compute $w = y_b^x \bmod p$
   (c) Derive key $k = G(w)$ using hash function G
   (d) Compute $r = H(m, w)$ using hash function H
   (e) Compute $s = \frac{x}{1+x_a \cdot r} \bmod q$ /* Specific to 2nd Example */
   (f) Encrypt the message $c = E_k(m)$ using symmetric encryption
4. Alice writes the signcrypted message (c, r, s) to **"Signcryption.txt"**.
5. Alice writes the signcrypted message to **"SigncryptedMessage.txt"** for Bob to read.

Terminal:

```
Compile the signers part with the following command:
gcc Sign.c -lcrypto -o Alice
Run your code with the following commands:
./Alice Message#.txt Seed#.txt
```

2.3 Unsigncryption: Bob [15 pt]

Task Description

Implement the unsigncryption algorithm in the file `Verify.c`. You **ONLY** need to implement the `Unsigncrypt()` function. This function is responsible for verifying and decrypting a signcrypted message sent from Alice to Bob.

Function Signature

```
int Unsigncrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM *r, BIGNUM *s,
                const unsigned char *ciphertext, size_t ciphertext_len,
                BIGNUM *bob_private_key, BIGNUM *alice_public_key,
                unsigned char **message, size_t *message_len)
```

Parameters

- `p, q, g`: System parameters
- `r, s`: Signature components from the signcryption
- `ciphertext`: The encrypted message
- `bob_private_key`: Bob's private key (x_b)
- `alice_public_key`: Alice's public key (y_a)
- `message`: Output parameter for the decrypted message
- `message_len`: Output parameter for the length of the decrypted message
- **Return value:** 1 if verification succeeds, 0 if it fails

Example Implementation Structure

```
int Unsigncrypt(BIGNUM *p, BIGNUM *q, BIGNUM *g, BIGNUM *r, BIGNUM *s,
                const unsigned char *ciphertext, size_t ciphertext_len,
                BIGNUM *bob_private_key, BIGNUM *alice_public_key,
                unsigned char **message, size_t *message_len)
{
    // 1. Compute w = (g * y_a^r)^(s*x_b) mod p

    // 2. Derive encryption key k = Hash_G(w)

    // 3. Decrypt the ciphertext using D_k(c)

    // 4. Compute r' = Hash_H(m, w)

    // 5. Verify that r = r' and return result
}
```

2.3 Unsigncryption: Bob – Instructions

Input Arguments

- System parameters file **"Params.txt"**.
- Bob's key pair file **"BobKeys.txt"**.
- Alice's public key file **"AlicePublicKey.txt"**.

Bob performs the unsigncryption as follows:

1. Bob reads the signcrypted message (c, r, s) from **"SigncryptedMessage.txt"**.
2. Bob reads the system parameters and key information.
3. Bob performs the unsigncryption algorithm:
   (a) Compute $w = (g \cdot y_a^r)^{s \cdot x_b} \bmod p$ /* Specific to 2nd Example */
   (b) Derive key $k = G(w)$ using hash function G
   (c) Decrypt the message $m = D_k(c)$ using symmetric decryption
   (d) Verify that $r = H(m, w)$ using hash function H
4. If verification succeeds, Bob writes "Signature is Valid" to **"Verification.txt"** and the decrypted message to **"DecryptedMessage.txt"**.
5. If verification fails, Bob writes "Verification Failed" to **"Verification.txt"**.

Terminal:

```
Compile the verifier part with the following command:
gcc Verify.c -lcrypto -o Bob
Run your code with the following commands:
./Bob
```

---

3 BPV Algorithm Questions [20 pt] (Due: *October 29th, 2026 – 11:59 pm*)

Question 1

The BPV (Boyko-Peinado-Venkatesan) algorithm is employed in cryptographic operations to enhance the efficiency of exponentiation by utilizing an offline precomputation phase.

(a) Describe how the BPV algorithm typically quantifies the *gain* in computational efficiency for the online phase compared to a standard square-and-multiply method. Express this gain in terms of the exponent length L (for the standard method's approximate $1.5L$ multiplications) and the BPV online phase's multiplication cost, T.

(b) Assume a system uses 4096-bit exponents ($L = 4096$). If the BPV algorithm is implemented with a window size of $K = 16$ bits, estimate the approximate **number of multiplications saved** in the online phase when using BPV compared to a standard square-and-multiply algorithm. Assume that the number of online multiplications T in BPV is approximately L/K.

Question 2

A cryptographic application uses the BPV algorithm with a base g and a fixed 1024-bit exponent ($L = 1024$). The system is deployed on a resource-constrained device with limited memory, allowing storage for at most **127 precomputed values**.

Given that BPV requires $2^K - 1$ precomputed values for a K-bit window:

(a) Determine the largest integer K that can be used without exceeding the memory limit.

(b) For your chosen K, compute the approximate number of online multiplications T.

(c) Compare this T to the standard square-and-multiply multiplication count ($\approx 1.5L$) and compute the percentage reduction in online multiplications.

Question 3

A specific cryptographic system implements exponentiation with 2048-bit exponents ($L = 2048$). A standard square-and-multiply approach would require about $1.5L$ multiplications for each exponentiation. The BPV algorithm is adopted with a precomputation window size of $K = 8$ bits, and its online phase is determined to require $T = 256$ multiplications.

Perform the following calculations:

(a) What is the approximate number of multiplications required by the **standard square-and-multiply** algorithm for one exponentiation?

(b) What is the approximate number of multiplications required in the **BPV offline phase** for $K = 8$?

(c) What is the approximate **total number of multiplications** for one BPV exponentiation (sum of offline and online phases)?

(d) Calculate the percentage of multiplication savings achieved by using the BPV algorithm compared to the standard method.

Question 4

A cryptographer is developing a time-sensitive cryptographic application where exponentiation of a 3072-bit exponent ($L = 3072$) must be completed in an online phase with a strict maximum of 300 multiplications. The BPV algorithm is being considered for its efficiency.

Given that the number of online multiplications T in BPV is approximately related to the exponent length L and window size K by $T \approx L/K$:

(a) Recommend an appropriate integer value for the window size K that satisfies the online multiplication constraint. Justify your choice with calculations.

(b) Based on your chosen K, what would be the approximate number of online multiplications T?

(c) Briefly explain the trade-offs a cryptographer must consider when choosing the window size K in BPV, particularly concerning the offline phase cost and storage requirements versus the online phase speed.

4 Extra Credit: Post-Quantum Cryptography & Efficiency Challenges [10 pt] (Due: *November 12th, 2026 – 11:59 pm*)

With the advent of quantum computing, many current public-key cryptosystems, including ECC, are vulnerable. Post-Quantum Cryptography (PQC) aims to develop new cryptographic algorithms resistant to quantum attacks.

(a) Research and identify one leading PQC algorithm that is being standardized or is a prominent candidate in the NIST PQC standardization process. Briefly describe the hard mathematical problem that underpins its security, analogous to the ECDLP for ECC.

(b) Based on your research, discuss one significant practical challenge that this chosen PQC algorithm faces compared to ECC, particularly in terms of computational efficiency or key/signature/ciphertext size. Provide a qualitative comparison (e.g., "PQC algorithm X typically has much larger key sizes than ECC for comparable security levels, leading to increased bandwidth usage.") and explain why this challenge exists, relating it back to the underlying mathematical problem or algorithmic structure. You do not need to provide exact multiplication counts, but demonstrate an understanding of the efficiency trade-offs.

Notes

- **Refer to the OpenSSL tutorial for details on how to use the PRNG function and BIGNUM operations.**
- You can also refer to the OpenSSL library documentation `https://docs.openssl.org/master/` to learn how to use the BIGNUM operations (It does not cover the PRNG).
- In grading on our side, we will test your code with multiple different seeds and messages, and also with a wrong signature. Hence, make sure your program does as instructed.
- The code that throws errors or does not successfully compile/run on our side, receives zero credit (It doesn't matter if it works on your computer!).
- Compile and run your code for functionality with provided commands. You can check your code for correctness with the script to verify your solution for the whole programming project. Please, only submit your code if it is functional.
- In order to use the script, just put your codes, the rest of the provide files along with the "RunTestVectors.sh" in one folder and run the following command in the terminal: ./RunTestVectors.sh

5 Rubric

- **Signcryption 1st Example Implementation** [50 pt]
  - Key Generation [15 pt]
  - Signcryption Process [20 pt]
  - Verification and Decryption [15 pt]
- **Signcryption 2nd Example Implementation** [30 pt]
  - Key Generation [Same as 1st Example]
  - Signcryption Process [15 pt]
  - Verification and Decryption [15 pt]
- **BPV Algorithm Questions** [20 pt]
- **Extra Credit** [10 pt]